From C++ to the Browser

The WebAssembly Compiler Pipeline

Kyungtak · LLVM integration & release, Google Toolchains

JSConf Mexico 2026

Follow one line of C++ all the way into a JavaScript engine.

THE SETUP

You already ship WebAssembly

Figma

Vector editor — C++ in the tab

Photoshop Web

Adobe's C++ core, compiled

AutoCAD Web

Decades of C++, in a browser

Google Earth

3D engine as WebAssembly

Browser games

Unity / Unreal → WASM

Your npm deps

Shiki runs Oniguruma as WASM

Real production apps your users open every day — compiled code running next to your JavaScript.

How does a line of C++ run in a browser safely and fast?

Not “it compiles to WASM.” Step by step — what actually happens to it.

THE PROBLEM

Why not just JavaScript?

Heavy compute

Codecs, physics, image filters. A good JIT can match a tight loop; it can't promise to, and it has no SIMD.

A compile target

Other languages (C++, Rust) needed somewhere to land.

Predictable speed

GC pauses and JITs make worst-case performance hard to promise.

JavaScript is brilliant glue. We needed a second contract for the heavy lifting.

DEFINITION

What WebAssembly actually is

A portable, sandboxed, statically-typed **binary instruction format**.

A compile target — not a language.

Rarely written by hand

It's an assembly the engine agrees to run.

Lives beside your JS

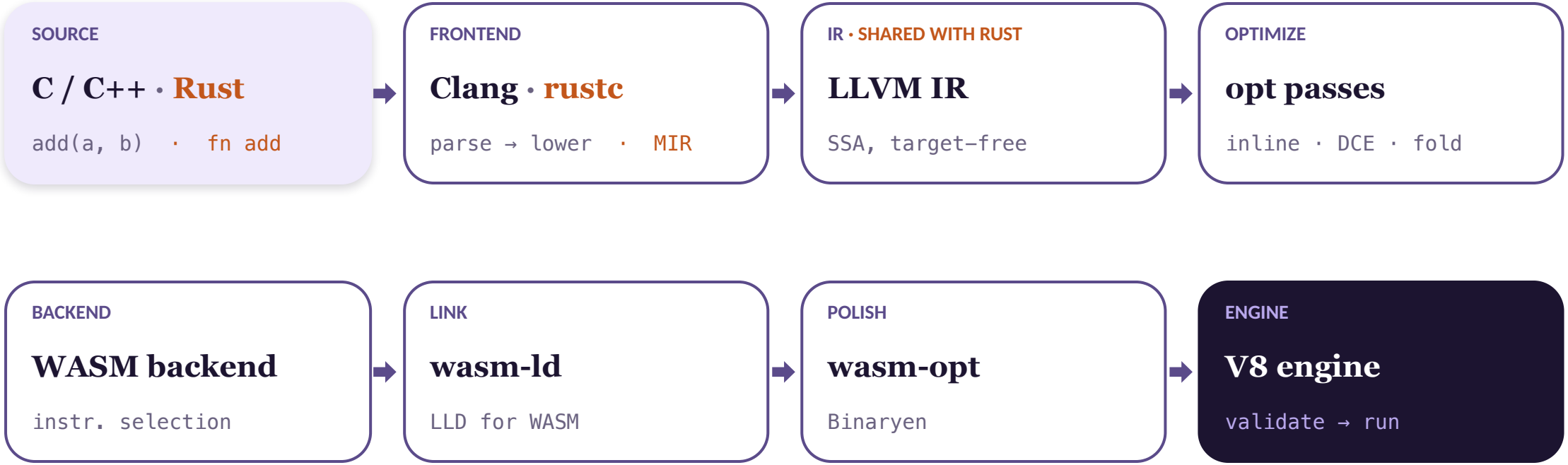
Same engine, same page. JS can read its memory directly.

“A CPU shaped like a contract”

Low-level, but verifiable and safe.

THE MAP

The pipeline, whole



The source is for humans. The IR is for the optimizer. The .wasm is the product the engine runs.
Rust takes the same road: rustc replaces Clang and lowers to the same LLVM IR; wasm-ld is rust-lld. From IR on, every box is shared.

OUR SPECIMEN

The line of code we're following

```
int add(int a, int b) {
    return a * 2 + b;
}
```

Trivial on purpose. Watch what the pipeline does to it.

STEP 1 · FRONTEND

Clang turns C++ into LLVM IR

SOURCE (for humans)

```
int add(int a, int b) {
  return a * 2 + b;
}
```

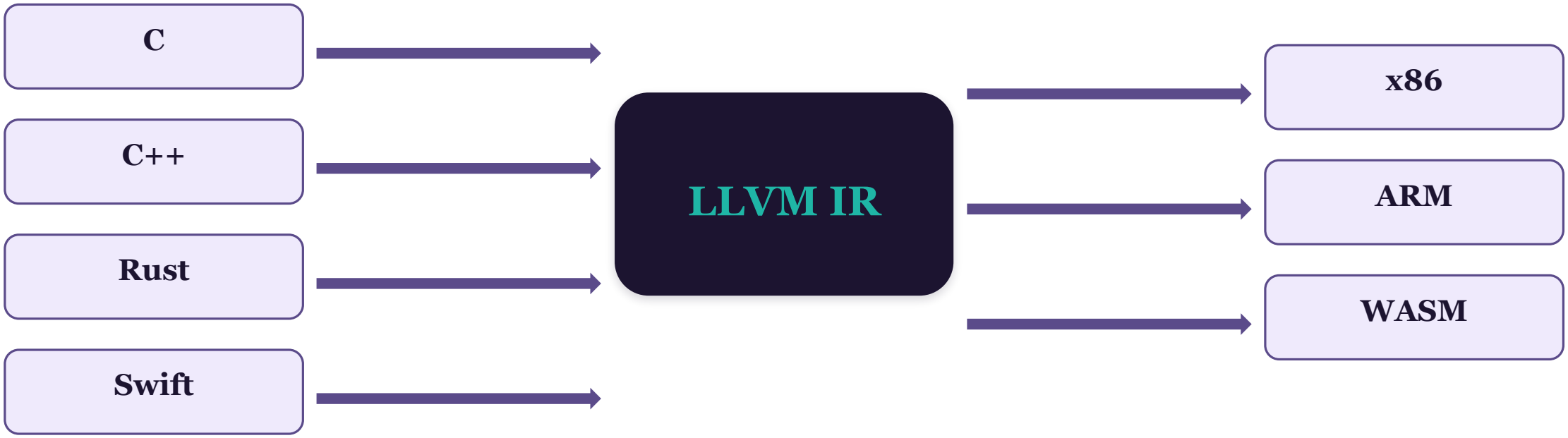
LLVM IR at -O0 (for the optimizer)

```
define i32 @_Z3addii(i32 %0, i32 %1) {
  %3 = alloca i32
  %4 = alloca i32
  store i32 %0, ptr %3
  store i32 %1, ptr %4
  %5 = load i32, ptr %3
  %6 = mul nsw i32 %5, 2
  %7 = load i32, ptr %4
  %8 = add nsw i32 %6, %7
  ret i32 %8
}
```

*At -O0 Clang is literal: variables live in memory and a*2 is still a mul. (C++ mangled add → _Z3addii.)*

STEP 2 · WHY IR

IR is the real product, not the source



M languages × N targets becomes M + N. That decoupling is why the whole ecosystem scales.

STEP 3 · OPTIMIZE

Optimization, intuitively

Dead code elimination

Delete computations no one uses.

```
x = f();  // x never read
→ removed
```

Inlining

Paste small functions into the caller.

```
add(2,3) → 2*2+3
→ then folded to 7
```

Constant folding

Compute constants at build time.

```
2 * 21
→ 42
```

THE PASSES THAT DID THE WORK IN THIS TALK

`mem2reg / SROA`	locals become SSA values: the 9 → 3 on slide 11	`loop unroll`	unrolled bodies, fewer branches (the hash)
`instcombine`	per-instruction algebra: mul 2 → shl 1	`loop / SLP vectorize`	scalar loops into SIMD lanes, when it can
`GVN / CSE`	compute once, reuse the value	`devirtualization`	virtual → direct call when the type is known
`LICM`	hoist loop-invariant work out of the loop	`IPSCCP / inlining`	constants and bodies flow across functions

Passes over the IR. Same meaning — less machine. -O2 / -O3 / -Oz pick the presets.

STEP 3 · OUR SPECIMEN

What -O2 did to add()

LLVM IR at -O2

```
define i32 @_Z3addii(i32 %0, i32 %1) {
  %3 = shl nsw i32 %0, 1
  %4 = add nsw i32 %3, %1
  ret i32 %4
}
```

WHAT CHANGED

```
alloca / store / load  →  gone
  mem2reg: values live in registers

mul %a, 2  →  shl %a, 1
  instcombine: a shift is cheaper

9 instructions  →  3
```

*Same meaning, a third of the work. The a*2 → shift trick belongs to the optimizer, not the frontend.*

STEP 4 · EMIT + LINK

From IR to a .wasm binary

1	`WASM backend`	Instruction selection: IR → WebAssembly ops.
2	`wasm-ld`	LLVM's linker (LLD) for WASM: resolves symbols, lays out memory.
3	`.wasm`	A real, portable binary artifact.
4	`wasm-opt`	Binaryen polishes the whole module for size + speed.

Emscripten orchestrates all of this and adds the JS glue — runtime, libc shims, filesystem.

STEP 4 · POLISH

wasm-opt: what Binaryen does

Locals, coalesced

LLVM's backend spends a local per value. Binaryen reuses them and keeps values on the stack.

```
local.set 2
local.get 2
i32.add
→ i32.add, no local
```

Peepholes on WASM ops

Rewrites LLVM never sees, because the shapes only exist after instruction selection.

```
if c (then 1)
     (else 0)
→ c
  (c is already 0 or 1)
```

Whole-module view

Every function at once: identical bodies merge, and what no export can reach is gone.

```
func $f … func $g
  same bytes
→ one function
→ unreachable: removed
```

Memory packing

Data segments full of zeros vanish. Linear memory starts zeroed anyway.

```
(data "\00 … \00 \01")
→ (data "\01") at end
→ 60 KB off the wire
```

WHY IT CHANGED NOTHING HERE · MEASURED WITH BINARYEN 133 ON THIS DEMO

-O3 module: 2,374 → 2,383 bytes, 558 → 558 instructions. Only the local numbers moved; the 9 extra bytes are a metadata section.
LLVM already had the whole program: one source file, no libc, explicit exports. Inlining, DCE and the locals were settled at -O3.
-O0 module: 3,432 → 2,644 bytes, 23% smaller. wasm-opt redoes much of the optimizer's work after the fact, but not all: -O3 is 2,374.
A real Emscripten app: dozens of objects, libc, runtime glue. There the whole-module view typically takes off 10 to 20%.

STEP 4 · OUR SPECIMEN

add() as WebAssembly

WASM TEXT (wasm2wat add.wasm)

```
(func $add (param i32 i32) (result i32)
  local.get 0
  i32.const 1
  i32.shl
  local.get 1
  i32.add)
```

READ IT AS A STACK

```
push a
push 1
shift left      →  a*2
push b
add             →  a*2 + b
```

WASM is a stack machine. The whole .wasm file: about 200 bytes, 14 of them code. The exact count depends on which Clang built it.

STEP 5 · THE ENGINE

Inside the JavaScript engine

Validate

Type-check every instruction before it runs. Safe with bounds-checked memory and typed indirect calls.

Compile — twice

Liftoff compiles everything for an instant start; only hot functions get the optimizing tier.

Linear memory

One sandboxed ArrayBuffer. Every load and store is bounds-checked.

The boundary

Only numbers cross directly. Next slide: what that means.

Validate → Liftoff → hot code tiers up in the background → run. Safe by construction: types, bounds, signatures.

THE BOUNDARY

Crossing JS ↔ WASM

YOUR SIDE (JavaScript)

```
const { instance } = await WebAssembly
  .instantiateStreaming(fetch("add.wasm"));

instance.exports.add(20, 2);   // 42

const { memory } = instance.exports;
// a view onto WASM memory, not a copy
const px = new Uint8ClampedArray(
  memory.buffer, ptr, w * h * 4);
```

WHAT BITES

```
Only numbers cross directly.
  (i64 arrives as a BigInt)
Strings & arrays = pointer + length.
  Encode into memory yourself.
memory.grow() detaches old views.
  Re-create them after growing.
Cross once, work a lot.
  Chatty calls eat the speedup.
```

Emscripten and wasm-bindgen write this glue for you. It's still worth knowing what it does.

WHY IT WORKS

Four words, demystified

Stack machine

No registers. Each instruction pops its inputs off a typed stack and pushes its result: push a, push 1, shift, push b, add. Simple enough that the engine checks every instruction in one pass before running any.

Linear memory

The module's whole world is one flat, growable array of bytes. A C++ pointer is just an index into it. Every load and store is bounds-checked, and JavaScript sees the same bytes as an ArrayBuffer.

Sandbox

Nothing is ambient. No DOM, no files, no network, no other memory. The module can only call the functions JavaScript handed in, and it runs only after validation says every instruction is well-typed.

Portable binary

One format for every CPU and every engine. The toolchain decides what the bytes say; the engine only decides how fast they run. The same bytes give the same result on any machine.

Checked once, boxed into one array, no exit but the imports, same bytes everywhere: why a browser can run a binary it has never seen.

LIVE DEMO

Let's watch it run

1 · The pipeline, live

add() ⟶ a template that disappears ⟶ constexpr ⟶ a virtual call. Source, IR at -O0, IR at -O2, WASM text and the bytes, for each.

2 · The optimizer, measured

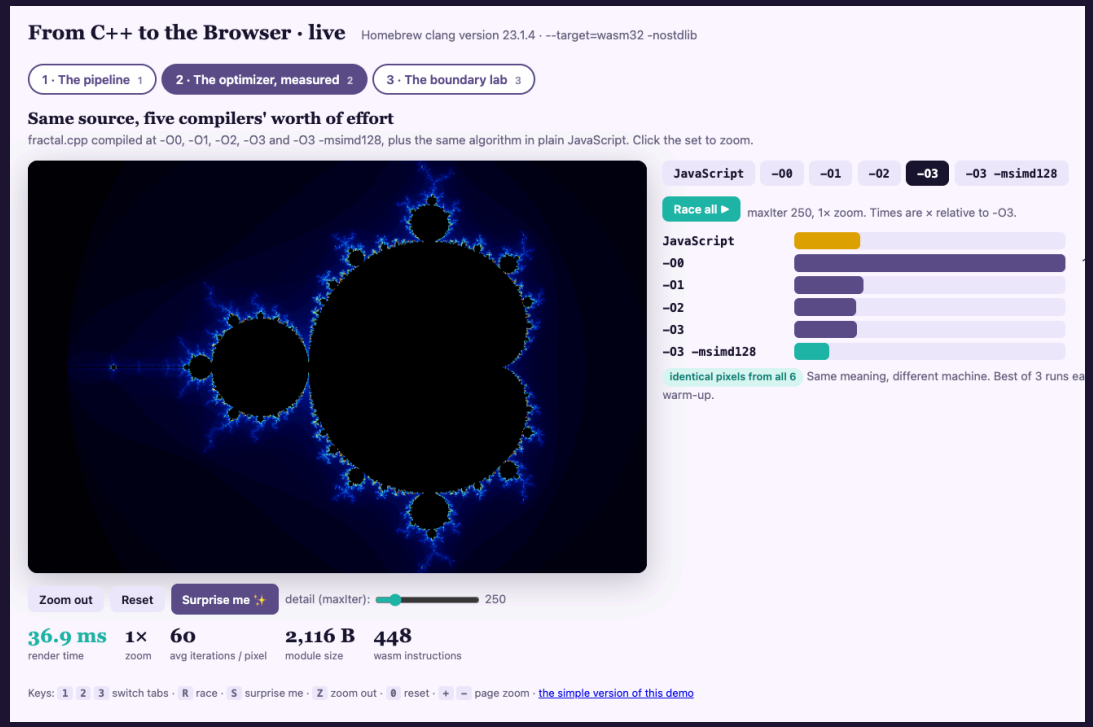
Race JavaScript against the same C++ at -O0, -O1, -O2, -O3 and -O3 with SIMD. Same source, same engine, identical pixels.

3 · The boundary lab

add(20, 2) · an i64 arrives as a BigInt · a string is pointer + length · memory.grow() detaches every view.

4 · Where JavaScript can't tie

A 64-bit hash four ways: BigInt, 32-bit halves by hand,
C++ at -O0 and -O3. Same hash from all four; the times differ.



The race tab: six builds of one C++ file, identical pixels, the milliseconds tell the story.

`kevinwkt.github.io/from-cpp-to-the-browser/demo` `keys: 1 2 3 4 tabs · R race (2 or 4) · S surprise me · Z zoom out · 0 reset`

ACT II

The Machine Room

Everything so far was the happy path.

Now: what it takes to run this pipeline for thousands of engineers.

MY DAY JOB

I take upstream LLVM and land it inside Google.

Integration

Build the open-source compiler against a giant internal codebase.

Release trains

Move everyone to a newer compiler without breaking anyone.

Reproducibility

Same inputs → same bits, on every machine, forever.

Upstream commit access

Fix problems at the source and push them back.

WAR STORY

A “correct” compiler upgrade.
A handful of binaries subtly wrong.

1	**All green**	Every test passed. The upgrade looked completely clean.
2	**Confidently wrong**	Not a crash — a few binaries produced subtly wrong output.
3	**Bisect at scale**	Narrow a range of upstream changes across many targets.
4	**The point: trust**	At scale, “it compiles” isn't the bar. Correctness you can trust is.

AT SCALE

Reproducibility turns a tool into infrastructure

Same inputs → the same bits, every time.

Hermetic builds

Sealed inputs; nothing leaks in from the machine.

Pinned toolchains

Everyone on a known, exact compiler version.

Bit-for-bit

Two builds of the same code are byte-identical.

It's what lets thousands of engineers trust the toolchain enough to never think about it.

BE HONEST

The sharp edges

Debugging is harder

DWARF-in-wasm and source maps exist, but trail native.

The boundary costs

Crossing JS ↔ WASM and copying data isn't free.

Tooling still churns

The spec moves fast; “stable” is recent, not ancient.

Not always a win

For light DOM work, plain JS is simpler and often faster.

Chatty JS↔WASM designs lose exactly the speed you came for. Fit matters more than a leaderboard.

WHERE IT'S GOING

WASM is growing beyond the browser

WASI 0.2 — stable	Jan 2024. Plus the Component Model: typed cross-language interop.
WASI 0.3 — async	June 2026. Native async via futures and streams; in Wasmtime and jco.
Server & edge	Wasmtime, Node, Deno and edge platforms run the same binaries.

And Wasm 3.0 (Sep 2025) added GC, so Kotlin, Dart and Java now compile to it too.

We started with one line of C++.

Lowered, optimized, emitted, linked, polished, validated, compiled — twice — and now running in a sandbox in your browser.

The source was never the product.
The toolchain is the product.

Thank you.

Talk to me about compilers and toolchains.

Kyungtak

LLVM integration & release, Google Toolchains


```
Repo + slides:  github.com/kevinwkt/from-cpp-to-the-browser
Live demo:      kevinwkt.github.io/from-cpp-to-the-browser/demo
Recording posted after the talk.
```